

New-API 集成 GitHub Copilot 方案

综合参考 [cc-switch](#)、[openclaw](#)、[copilot-api](#) 三个项目的实现，面向 new-api 服务端 API Gateway 场景设计。

目录

1. [关于 GitHub Copilot SDK 的判断](#一关于-github-copilot-sdk-的判断)
2. [三个参考项目横向对比](#二三个参考项目横向对比)
3. [整体架构设计](#三整体架构设计)
4. [核心实现细节](#四核心实现细节)
 - [认证层：GitHub Token 管理](#41-认证层github-token-管理)
 - [Token Exchange 层](#42-token-exchange-层最核心)
 - [动态 Base URL 解析](#43-动态-base-url-解析)
 - [Copilot 请求 Headers（完整版）](#44-copilot-请求-headers完整版)
 - [动态 VSCode 版本获取](#45-动态-vscode-版本获取)
 - [协议路由与转换](#46-协议路由与转换)
 - [Anthropic → OpenAI 请求转换](#47-anthropic--openai-请求转换messages-端点)
 - [OpenAI → Anthropic 响应转换](#48-openai--anthropic-响应转换)
 - [流式响应状态机转换](#49-流式响应状态机转换)
 - [动态模型列表与 max_tokens 填充](#410-动态模型列表与-max_tokens-填充)
 - [new-api 渠道集成结构](#411-new-api-渠道集成结构)
5. [内置模型列表](#五内置模型列表)
6. [实施优先级路线图](#六实施优先级路线图)
7. [关键注意事项](#七关键注意事项)
8. [参考资料](#八参考资料)

一、关于 GitHub Copilot SDK 的判断

先回答核心问题：在 new-api 中用 @github/copilot-sdk 还是纯 HTTP 直调？

维度	@github/copilot-sdk	纯 HTTP 直接调用
本质	封装本地 Copilot Language Server (Node.js 子进程)	直接调用 Copilot REST API
适用场景	桌面工具、VS Code 扩展	服务端代理、API Gateway
代表项目	cc-switch、copilot-sdk-proxy	openclaw、copilot-api (最佳参考)
缺点	要求宿主机安装 Node.js；需管理子进程生命周期；不适合高并发多用户	需自行实现 token exchange 和 IDE 头注入
结论	❌ 不推荐用于 new-api	✅ 推荐

结论说明：cc-switch 和 copilot-sdk-proxy 使用 SDK 是因为它们是桌面端本地工具。new-api 是多用户服务端 API Gateway，openclaw 和 copilot-api 的纯 HTTP 方案才是正确参考范本，其中 copilot-api 架构最清晰、细节最完整，是三个项目中对 new-api 借鉴价值最高的。

二、三个参考项目横向对比

特性	cc-switch	openclaw	copilot-api	new-api 采纳
Token Exchange	✅	✅	✅	✅

动态 Base URL (proxy-ep 解析)	✗	✓	✗ (按类型拼接)	✓ 两种都实现
多账号类型 (business/enterprise)	✗	✗	✓	✓ 借鉴
Token 刷新策略	被动 (过期检查)	被动 (过期检查)	✓ 主动 (refresh_in 驱动)	✓ 借鉴
VSCode 版本获取	硬编码	硬编码	✓ 动态从 AUR 拉取	✓ 借鉴
x-request-id	✗	✗	✓ 每次 UUID	✓ 必须加
x-vscode-user-agent-library-version	✗	✗	✓ electron-fetch	✓ 必须加
X-Initiator Header	✗	✗	✓ user / agent	✓ 建议加
Anthropic Messages 端点 (/v1/messages)	✗	✗	✓ 完整双向转换	✓ 借鉴
模型名规范化 (Claude 子版本截断)	✗	✗	✓	✓ 必须
动态 max_tokens 填充	✗	固定 8192	✓ 从模型 capabilities 读取	✓ 借鉴
Embedding 支持	✗	✓	✓	✓ 支持
动态模型列表 (/v1/models 实时拉取)	✗	✗	✓	✓ 借鉴
使用量查询	✗	✓	✓	✓
请求级别限流	✗	✗	✓	new-api 已有

三、整体架构设计

Code block

```
1  用户请求
2  |
3  |—— POST /v1/chat/completions  (OpenAI 格式, 透传)
4  |
5  |    |—— ① 取 GitHub Token → ② Token Exchange (带缓存)
6  |    |    → ③ 解析 Base URL → ④ 注入完整 Headers
7  |    |    → ⑤ 转发到 Copilot /chat/completions
8  |    |    → ⑥ 流式/非流式透传返回
9  |
10 |—— POST /v1/messages          (Anthropic 格式, 需协议转换)
11 |
12 |    |—— ① 同上认证流程
13 |    |    → ② translateToOpenAI(anthropicPayload)
14 |    |    → ③ 转发到 Copilot /chat/completions
15 |    |    → ④ translateToAnthropic(response) / 流式状态机转换
16 |    |    → ⑤ 返回 Anthropic 格式响应
17 |
18 |—— POST /v1/embeddings        (Embedding, 透传)
19 |    |—— → Copilot /embeddings
20 |
21 |—— GET /v1/models             (动态模型列表)
22 |    |—— → Copilot /models (缓存 30 分钟)
23
24 认证层 (渠道级, 带缓存):
25  Channel.APIKey (GitHub Token)
26  → GET api.github.com/copilot_internal/v2/token
27  → Copilot Token (按 refresh_in 主动刷新)
28  → 解析 proxy-ep → 动态 Base URL
```

四、核心实现细节

4.1 认证层：GitHub Token 管理

new-api 面向多用户，**不需要实现 Device Flow**，由用户自行填入 GitHub Token。

渠道配置字段：

字段	说明
Channel Type	GitHub Copilot （新增类型编号，建议 45）
API Key	GitHub Token （ ghp_ / ghu_ / gho_ 前缀均支持）
Base URL	可选填账号类型（ individual / business / enterprise ），留空默认 individual

Token 来源优先级：

优先级	来源	说明
1	Channel API Key 字段	new-api 数据库中存储的 token，优先使用
2	COPILOT_GITHUB_TOKEN 环境变量	Copilot 专用
3	GH_TOKEN 环境变量	GitHub CLI token
4	GITHUB_TOKEN 环境变量	标准 GitHub token

支持的 Token 格式：

Code block

- 1 ghp_xxxx → GitHub Personal Access Token (PAT) ，最推荐，权限可精确控制
- 2 ghu_xxxx → Device Flow OAuth Token (第三方工具登录后产生)
- 3 gho_xxxx → OAuth App Token

4.2 Token Exchange 层（最核心）

综合三个项目的实现，Token Exchange 接口信息如下：

Code block

- 1 GET https://api.github.com/copilot_internal/v2/token
- 2 Authorization: Bearer {github_token}
- 3 + Copilot IDE 标识 Headers (见 4.4)

响应格式（完整版，来自 copilot-api）：

Code block

```
1 {
2   "token": "tid=xxx;st=xxx;proxy-
  ep=proxy.individual.githubcopilot.com;exp=xxx;...",
3   "expires_at": 1234567890,
4   "refresh_in": 1800
5 }
```

`refresh_in`（单位：秒）是服务端建议的刷新闻隔，应在 `refresh_in - 60` 秒时主动刷新，而不是等到过期。这是 copilot-api 独有发现，比 openclaw 的被动过期检查更合理。

Go 实现参考：

Code block

```
1  // relay/channel/copilot/token.go
2
3  package copilot
4
5  import (
6      "encoding/json"
7      "fmt"
8      "net/http"
9      "sync"
10     "time"
11 )
12
13 const CopilotTokenURL = "https://api.github.com/copilot_internal/v2/token"
14 const DefaultCopilotBaseURL = "https://api.githubcopilot.com"
15
16 type CopilotTokenCache struct {
17     Token      string `json:"token"`
18     BaseURL    string `json:"base_url"`
19     ExpiresAt  int64  `json:"expires_at"` // Unix 毫秒时间戳
20     RefreshIn  int    `json:"refresh_in"` // 服务端建议刷新间隔 (秒)
21     UpdatedAt  int64  `json:"updated_at"`
22 }
23
24 // 全局内存缓存 (key: githubToken, value: *CopilotTokenCache)
25 // 生产环境可替换为 Redis, 实现多实例共享
26 var tokenCache sync.Map
27
28 func GetCopilotToken(githubToken string) (copilotToken string, baseURL string,
29 err error) {
30     // 1. 检查内存缓存, 提前 5 分钟过期 (安全边际)
31     if cached, ok := tokenCache.Load(githubToken); ok {
32         c := cached.(*CopilotTokenCache)
33         if time.Now().UnixMilli() < c.ExpiresAt-5*60*1000 {
34             return c.Token, c.BaseURL, nil
35         }
36     }
37
38     // 2. 发起 Token Exchange 请求
39     req, err := http.NewRequest("GET", CopilotTokenURL, nil)
40     if err != nil {
41         return "", "", err
42     }
43 }
```

```

42 req.Header.Set("Authorization", "Bearer "+githubToken)
43 req.Header.Set("Accept", "application/json")
44 InjectCopilotIdeHeaders(req, GetVSCodeVersion(), false, false)
45
46 resp, err := http.DefaultClient.Do(req)
47 if err != nil {
48     return "", "", err
49 }
50 defer resp.Body.Close()
51
52 if resp.StatusCode != 200 {
53     return "", "", fmt.Errorf("copilot token exchange failed: HTTP %d",
resp.StatusCode)
54 }
55
56 var result struct {
57     Token      string `json:"token"`
58     ExpiresAt  int64  `json:"expires_at"` // 秒级时间戳
59     RefreshIn  int    `json:"refresh_in"` // 服务端建议刷新间隔
60 }
61 if err = json.NewDecoder(resp.Body).Decode(&result); err != nil {
62     return "", "", err
63 }
64
65 expiresAtMs := result.ExpiresAt * 1000
66 resolvedBaseUrl := ResolveCopilotBaseUrl("", result.Token)
67
68 // 3. 写入内存缓存
69 cached := &CopilotTokenCache{
70     Token:      result.Token,
71     BaseURL:    resolvedBaseUrl,
72     ExpiresAt:  expiresAtMs,
73     RefreshIn:  result.RefreshIn,
74     UpdatedAt:  time.Now().UnixMilli(),
75 }
76 tokenCache.Store(githubToken, cached)
77
78 // 4. 启动主动刷新定时器 (copilot-api 的设计)
79 go scheduleTokenRefresh(githubToken, result.RefreshIn)
80
81 return result.Token, resolvedBaseUrl, nil
82 }
83
84 // scheduleTokenRefresh 按服务端建议的 refresh_in 主动刷新, 避免 token 过期
85 func scheduleTokenRefresh(githubToken string, refreshIn int) {
86     if refreshIn <= 60 {
87         return

```



```

88     }
89     // 提前 60 秒刷新
90     timer := time.NewTimer(time.Duration(refreshIn-60) * time.Second)
91     <-timer.C
92
93     newToken, newBaseURL, err := exchangeNewCopilotToken(githubToken)
94     if err != nil {
95         return
96     }
97
98     if cached, ok := tokenCache.Load(githubToken); ok {
99         c := cached.(*CopilotTokenCache)
100         c.Token = newToken
101         c.BaseURL = newBaseURL
102         c.UpdatedAt = time.Now().UnixMilli()
103         // 继续下一次刷新
104         go scheduleTokenRefresh(githubToken, c.RefreshIn)
105     }
106 }
107
108 // exchangeNewCopilotToken 仅做网络请求，不操作缓存（供定时刷新内部调用）
109 func exchangeNewCopilotToken(githubToken string) (string, string, error) {
110     req, _ := http.NewRequest("GET", CopilotTokenURL, nil)
111     req.Header.Set("Authorization", "Bearer "+githubToken)
112     req.Header.Set("Accept", "application/json")
113     InjectCopilotIdeHeaders(req, GetVSCodeVersion(), false, false)
114
115     resp, err := http.DefaultClient.Do(req)
116     if err != nil {
117         return "", "", err
118     }
119     defer resp.Body.Close()
120
121     var result struct {
122         Token      string `json:"token"`
123         ExpiresAt  int64  `json:"expires_at"`
124         RefreshIn  int    `json:"refresh_in"`
125     }
126     json.NewDecoder(resp.Body).Decode(&result)
127     return result.Token, ResolveCopilotBaseURL("", result.Token), nil
128 }

```

4.3 动态 Base URL 解析

综合 openclaw 的 `proxy-ep` 解析和 copilot-api 的账号类型拼接，实现双重策略：

Code block

```
1 // relay/channel/copilot/token.go
2
3 // ResolveCopilotBaseURL 双重策略解析实际 API Base URL
4 // 策略1 (openclaw) : 从 token 内嵌的 proxy-ep 字段解析 (精确, 推荐)
5 // 策略2 (copilot-api) : 按账号类型拼接 (fallback)
6 func ResolveCopilotBaseURL(accountType string, token string) string {
7     // 策略1: 解析 token 内嵌的 proxy-ep
8     // token 格式: "tid=xxx;st=xxx;proxy-
9     // ep=proxy.individual.githubcopilot.com;..."
10    if url := deriveCopilotBaseURLFromToken(token); url != "" {
11        return url
12    }
13
14    // 策略2: 按账号类型拼接 (copilot-api 的方式)
15    switch accountType {
16    case "business":
17        return "https://api.business.githubcopilot.com"
18    case "enterprise":
19        return "https://api.enterprise.githubcopilot.com"
20    default: // "individual" 或空
21        return DefaultCopilotBaseURL
22    }
23
24    // deriveCopilotBaseURLFromToken 解析 token 中的 proxy-ep 字段
25    func deriveCopilotBaseURLFromToken(token string) string {
26        for _, part := range strings.Split(token, ";") {
27            kv := strings.SplitN(strings.TrimSpace(part), "=", 2)
28            if len(kv) != 2 || strings.ToLower(strings.TrimSpace(kv[0])) != "proxy-
29            ep" {
30                continue
31            }
32            host := strings.TrimSpace(kv[1])
33            host = strings.TrimPrefix(host, "https://")
34            host = strings.TrimPrefix(host, "http://")
35            host = strings.ToLower(host)
36            // proxy.xxx.com → api.xxx.com (openclaw 发现的关键转换)
37            if strings.HasPrefix(host, "proxy.") {
38                host = "api." + host[6:]
39            }
40        }
41        return host
42    }
43 }
```

```
38         }
39         if host != "" {
40             return "https://" + host
41         }
42     }
43     return ""
44 }
```

各账号类型对应 Base URL（copilot-api 实测）：

Account Type	Base URL
<code>individual</code> （默认）	<code>https://api.githubcopilot.com</code>
<code>business</code>	<code>https://api.business.githubcopilot.com</code>
<code>enterprise</code>	<code>https://api.enterprise.githubcopilot.com</code>

4.4 Copilot 请求 Headers（完整版）

这是三个项目综合后最完整的 Headers 清单，缺少任何关键 Header 都可能导致 403 或异常行为：

Code block

```
1 // relay/channel/copilot/headers.go
2
3 package copilot
4
5 import (
6     "net/http"
7     "github.com/google/uuid"
```

```

8 )
9
10 const (
11     CopilotVersion          = "0.26.7"
12     EditorPluginVersion     = "copilot-chat/" + CopilotVersion
13     UserAgent               = "GitHubCopilotChat/" + CopilotVersion
14     GithubAPIVersion       = "2025-04-01"
15     VSCodeUserAgentLibrary = "electron-fetch"
16 )
17
18 // InjectCopilotIdeHeaders 注入 Copilot API 所有必要的 IDE 标识 Headers
19 // 综合三个项目 (cc-switch / openclaw / copilot-api) 的完整实现
20 func InjectCopilotIdeHeaders(req *http.Request, vsCodeVersion string,
    isAgentCall bool, hasImages bool) {
21     // === 基础认证与内容类型 ===
22     req.Header.Set("Content-Type", "application/json")
23     req.Header.Set("Accept", "application/json")
24
25     // === IDE 标识 (核心, 缺失会被拒绝) ===
26     req.Header.Set("editor-version", "vscode/"+vsCodeVersion) // 动态版本
27     req.Header.Set("editor-plugin-version", EditorPluginVersion)
28     req.Header.Set("user-agent", UserAgent)
29     req.Header.Set("copilot-integration-id", "vscode-chat")
30     req.Header.Set("openai-intent", "conversation-panel")
31
32     // === API 版本与追踪 ===
33     req.Header.Set("x-github-api-version", GithubAPIVersion)
34     req.Header.Set("x-request-id", uuid.NewString()) // 每次请求
    唯一 (copilot-api 发现)
35
36     // === VSCode 运行时标识 (copilot-api 独有发现, 模拟 VSCode 内置 fetch 库) ===
37     req.Header.Set("x-vscode-user-agent-library-version",
    VSCodeUserAgentLibrary)
38
39     // === 请求来源区分 (copilot-api 独有发现, 可能影响计费/限流) ===
40     // 消息历史中含有 assistant/tool 角色说明是 agent 发起的多轮对话
41     if isAgentCall {
42         req.Header.Set("X-Initiator", "agent")
43     } else {
44         req.Header.Set("X-Initiator", "user")
45     }
46
47     // === 多模态 (视觉) ===
48     if hasImages {
49         req.Header.Set("copilot-vision-request", "true")
50     }
51 }

```

```

52
53 // BuildCopilotRequestHeaders 构建完整的 Copilot API 请求 Headers (含
    Authorization)
54 func BuildCopilotRequestHeaders(copilotToken, vscodeVersion string,
    isAgentCall, hasImages bool) http.Header {
55     req := &http.Request{Header: make(http.Header)}
56     req.Header.Set("Authorization", "Bearer "+copilotToken)
57     InjectCopilotIdeHeaders(req, vscodeVersion, isAgentCall, hasImages)
58     return req.Header
59 }
60
61 // HasAgentMessages 判断消息历史中是否包含 agent 消息 (用于设置 X-Initiator)
62 func HasAgentMessages(messages []Message) bool {
63     for _, msg := range messages {
64         if msg.Role == "assistant" || msg.Role == "tool" {
65             return true
66         }
67     }
68     return false
69 }

```

完整 Headers 字段说明：

Header	来源项目	值	重要性
Authorization	全部	Bearer {copilot_token}	● 必须
editor-version	全部	vscode/{动态版本}	● 必须
editor-plugin-version	全部	copilot-chat/0.26.7	● 必须
user-agent	全部	GitHubCopilotChat/0.26.7	● 必须
copilot-integration-id	全部	vscode-chat	● 必须
openai-intent	全部	conversation-panel	● 重要
x-github-api-version	全部	2025-04-01	● 重要

x-request-id	copilot-api	每次 uuid.NewString()	● 重要
x-vscode-user-agent-library-version	copilot-api	electron-fetch	● 建议
X-Initiator	copilot-api	user 或 agent	● 建议
copilot-vision-request	cc-switch	true （有图时）	● 有图必须

⚠ `editor-version` 和 `editor-plugin-version` 需要定期更新，可关注 [cc-switch CHANGELOG](#) 获取最新版本号，或使用下方的动态获取方案。

4.5 动态 VSCode 版本获取

copilot-api 独创的方案：从 Arch Linux AUR 仓库实时拉取 VSCode 最新版本，**无需手动维护版本号**：

Code block

```
1 // relay/channel/copilot/vscode-version.go
2
3 package copilot
4
5 import (
6     "context"
7     "io"
8     "net/http"
9     "regexp"
10    "sync"
11    "time"
12 )
13
14 const vsCodeFallbackVersion = "1.104.3"
15
16 var (
17     cachedVSCodeVersion string
```

```
18     vsCodeVersionCacheMu sync.RWMutex
19     vsCodeVersionCachedAt time.Time
20     vsCodeVersionCacheTTL = 24 * time.Hour // 每天刷新一次
21 )
22
23 // GetVSCodeVersion 获取最新 VSCode 版本号 (带缓存, 24h 更新一次)
24 func GetVSCodeVersion() string {
25     vsCodeVersionCacheMu.RLock()
26     if cachedVSCodeVersion != "" && time.Since(vsCodeVersionCachedAt) <
vsCodeVersionCacheTTL {
27         v := cachedVSCodeVersion
28         vsCodeVersionCacheMu.RUnlock()
29         return v
30     }
31     vsCodeVersionCacheMu.RUnlock()
32
33     version := fetchLatestVSCodeVersion()
34
35     vsCodeVersionCacheMu.Lock()
36     cachedVSCodeVersion = version
37     vsCodeVersionCachedAt = time.Now()
38     vsCodeVersionCacheMu.Unlock()
39
40     return version
41 }
42
43 // fetchLatestVSCodeVersion 从 AUR PKGBUILD 实时解析 VSCode 最新版本号
44 // 来源: https://aur.archlinux.org/cgit/aur.git/plain/PKGBUILD?h=visual-studio-
code-bin
45 // 失败时返回 fallback 版本 (copilot-api 的方案)
46 func fetchLatestVSCodeVersion() string {
47     ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
48     defer cancel()
49
50     req, err := http.NewRequestWithContext(ctx, "GET",
51         "https://aur.archlinux.org/cgit/aur.git/plain/PKGBUILD?h=visual-studio-
code-bin", nil)
52     if err != nil {
53         return vsCodeFallbackVersion
54     }
55
56     resp, err := http.DefaultClient.Do(req)
57     if err != nil {
58         return vsCodeFallbackVersion
59     }
60     defer resp.Body.Close()
61
```

```
62     body, err := io.ReadAll(resp.Body)
63     if err != nil {
64         return vsCodeFallbackVersion
65     }
66
67     re := regexp.MustCompile(`pkgver=([0-9.]+)`)
68     if m := re.FindSubmatch(body); m != nil {
69         return string(m[1])
70     }
71
72     return vsCodeFallbackVersion
73 }
```

4.6 协议路由与转换

new-api 对外暴露两个端点，内部统一转发到 Copilot OpenAI-兼容接口：

Code block

```
1  // relay/channel/copilot/adaptor.go
2
3  package copilot
4
5  type Adaptor struct{}
6
7  func (a *Adaptor) GetRequestURL(meta *meta.Meta) (string, error) {
8      githubToken := meta.APIKey
9      accountType := resolveAccountType(meta.ChannelConfig) // 从渠道配置读取
10
11      copilotToken, baseURL, err := GetCopilotToken(githubToken)
12      if err != nil {
13          return "", err
14      }
15
16      // 将短期 Copilot Token 注入到本次请求上下文
17      meta.Config.APIKey = copilotToken
18      meta.BaseURL = baseURL
19
20      switch meta.Mode {
```



```

21     case relaymode.ChatCompletions:
22         return baseURL + "/chat/completions", nil
23     case relaymode.Embeddings:
24         return baseURL + "/embeddings", nil
25     default:
26         return baseURL + "/chat/completions", nil
27     }
28 }
29
30 func (a *Adaptor) SetupRequestHeader(c *gin.Context, req *http.Request, meta
    *meta.Meta) error {
31     copilotToken := meta.Config.APIKey
32     vsCodeVersion := GetVSCodeVersion()
33     messages := extractMessagesFromContext(c)
34     isAgentCall := HasAgentMessages(messages)
35     hasImages := requestHasImages(c)
36
37     headers := BuildCopilotRequestHeaders(copilotToken, vsCodeVersion,
    isAgentCall, hasImages)
38     for key, values := range headers {
39         for _, value := range values {
40             req.Header.Set(key, value)
41         }
42     }
43     return nil
44 }

```

协议转换矩阵：

外部请求格式	new-api 路由	Copilot API 端点	请求转换	响应转换
OpenAI Chat	POST /v1/chat/completions	/chat/completions	直接透传 + 模型名规范化	直接透传
Anthropic Messages	POST /v1/messages	/chat/completions	Anthropic → OpenAI	OpenAI → Anthropic
OpenAI Embeddings	POST /v1/embeddings	/embeddings	透传	透传

4.7 Anthropic → OpenAI 请求转换（ /messages 端点）

完整参考 copilot-api 的 `non-stream-translation.ts`，这是最复杂也最重要的转换逻辑：

消息角色映射：

Anthropic 消息类型	→ OpenAI 消息	特殊处理
<code>system</code> （字符串）	<code>{role: "system", content: str}</code>	直接转换
<code>system</code> （TextBlock 数组）	<code>{role: "system", content: 合并文本}</code>	用 <code>\n\n</code> 拼接
<code>user</code> + 普通文本	<code>{role: "user", content: str}</code>	直接转换
<code>user</code> + <code>tool_result</code> block	<code>{role: "tool", tool_call_id: ...}</code>	必须排在用户消息之前
<code>user</code> + <code>image</code> block	<code>image_url</code> ContentPart	<code>data:{mime};base64,{data}</code>
<code>assistant</code> + 文本	<code>{role: "assistant", content: str}</code>	直接转换
<code>assistant</code> + <code>thinking</code> block	合并进 <code>content</code> 文本	OpenAI 无 thinking 概念
<code>assistant</code> + <code>tool_use</code> block	<code>tool_calls[]</code>	<code>input</code> 序列化为 JSON 字符串

Tool Choice 映射：

	→ OpenAI <code>tool_choice</code>
--	-----------------------------------

Anthropic tool_choice.type	
"auto"	"auto"
"any"	"required"
"tool" + name	{type: "function", function: {name}}
"none"	"none"

模型名规范化（copilot-api 独有发现，必须实现）：

Code block

```
1 // relay/channel/copilot/model.go
2
3 // NormalizeCopilotModelName 规范化模型名
4 // Copilot 不支持带子版本号的 Claude 模型名 (如 claude-sonnet-4-20250514)
5 func NormalizeCopilotModelName(modelId string) string {
6     lower := strings.ToLower(modelId)
7     // "claude-sonnet-4-20250514" / "claude-sonnet-4-5" → "claude-sonnet-4"
8     if strings.HasPrefix(lower, "claude-sonnet-4-") {
9         return "claude-sonnet-4"
10    }
11    // "claude-opus-4-5" → "claude-opus-4"
12    if strings.HasPrefix(lower, "claude-opus-4-") {
13        return "claude-opus-4"
14    }
15    return modelId
16 }
```

Go 实现（请求转换核心）：

Code block

```
1 // relay/channel/copilot/anthropic_to_openai.go
2
3 package copilot
```

```

4
5 // TranslateAnthropicToOpenAI 将 Anthropic Messages 请求转换为 OpenAI Chat
  Completions 请求
6 func TranslateAnthropicToOpenAI(req *AnthropicRequest) *OpenAIChatRequest {
7     return &OpenAIChatRequest{
8         Model:      NormalizeCopilotModelName(req.Model),
9         Messages:   translateMessages(req.Messages, req.System),
10        MaxTokens:  req.MaxTokens,
11        Stop:       req.StopSequences,
12        Stream:     req.Stream,
13        Temperature: req.Temperature,
14        TopP:       req.TopP,
15        Tools:      translateTools(req.Tools),
16        ToolChoice:  translateToolChoice(req.ToolChoice),
17    }
18 }
19
20 func translateMessages(msgs []AnthropicMessage, system interface{})
  []OpenAIMessage {
21     var result []OpenAIMessage
22
23     // 处理 system prompt
24     if systemMsg := handleSystemPrompt(system); systemMsg != nil {
25         result = append(result, *systemMsg)
26     }
27
28     for _, msg := range msgs {
29         switch msg.Role {
30             case "user":
31                 result = append(result, handleUserMessage(msg)...)
32             case "assistant":
33                 result = append(result, handleAssistantMessage(msg))
34         }
35     }
36     return result
37 }
38
39 func handleUserMessage(msg AnthropicMessage) []OpenAIMessage {
40     var result []OpenAIMessage
41
42     if blocks, ok := msg.Content.([]AnthropicContentBlock); ok {
43         // 工具结果块必须先处理 (tool_result → role: "tool")
44         for _, block := range blocks {
45             if block.Type == "tool_result" {
46                 result = append(result, OpenAIMessage{
47                     Role:      "tool",
48                     ToolCallID: block.ToolUseID,

```

```

49         Content:    flattenContent(block.Content),
50     })
51 }
52 }
53 // 再处理其他内容块
54 var otherBlocks []AnthropicContentBlock
55 for _, block := range blocks {
56     if block.Type != "tool_result" {
57         otherBlocks = append(otherBlocks, block)
58     }
59 }
60 if len(otherBlocks) > 0 {
61     result = append(result, OpenAIMessage{
62         Role:    "user",
63         Content: mapContentBlocks(otherBlocks),
64     })
65 }
66 } else {
67     result = append(result, OpenAIMessage{
68         Role:    "user",
69         Content: msg.Content,
70     })
71 }
72 return result
73 }
74
75 func handleAssistantMessage(msg AnthropicMessage) OpenAIMessage {
76     blocks, ok := msg.Content.([]AnthropicContentBlock)
77     if !ok {
78         return OpenAIMessage{Role: "assistant", Content: msg.Content}
79     }
80
81     var textParts []string
82     var toolCalls []OpenAIToolCall
83
84     for _, block := range blocks {
85         switch block.Type {
86             case "text":
87                 textParts = append(textParts, block.Text)
88             case "thinking":
89                 // thinking block 合并进文本 (OpenAI 无对应概念)
90                 textParts = append(textParts, block.Thinking)
91             case "tool_use":
92                 args, _ := json.Marshal(block.Input)
93                 toolCalls = append(toolCalls, OpenAIToolCall{
94                     ID:    block.ID,
95                     Type: "function",

```

```

96         Function: OpenAIFunctionCall{
97             Name:      block.Name,
98             Arguments: string(args),
99         },
100     })
101 }
102 }
103
104 content := strings.Join(textParts, "\n\n")
105 if len(toolCalls) > 0 {
106     return OpenAIMessage{
107         Role:      "assistant",
108         Content:    nullableString(content),
109         ToolCalls:  toolCalls,
110     }
111 }
112 return OpenAIMessage{Role: "assistant", Content: content}
113 }

```

4.8 OpenAI → Anthropic 响应转换

将 Copilot 返回的 OpenAI 格式响应转换为 Anthropic 格式（非流式）：

Code block

```

1  // relay/channel/copilot/openai_to_anthropic.go
2
3  package copilot
4
5  // TranslateOpenAIToAnthropic 将 OpenAI 响应转换为 Anthropic 格式（非流式）
6  func TranslateOpenAIToAnthropic(resp *OpenAIChatResponse) *AnthropicResponse {
7      var contentBlocks []AnthropicContentBlock
8      var stopReason string
9
10     for _, choice := range resp.Choices {
11         // 提取文本块
12         if choice.Message.Content != "" {
13             contentBlocks = append(contentBlocks, AnthropicContentBlock{
14                 Type: "text",

```

```

15         Text: choice.Message.Content,
16     })
17 }
18 // 提取工具调用块
19 for _, tc := range choice.Message.ToolCalls {
20     var input map[string]interface{}
21     json.Unmarshal([]byte(tc.Function.Arguments), &input)
22     contentBlocks = append(contentBlocks, AnthropicContentBlock{
23         Type: "tool_use",
24         ID:    tc.ID,
25         Name:  tc.Function.Name,
26         Input: input,
27     })
28 }
29 if choice.FinishReason != "" {
30     stopReason = mapOpenAISTopReasonToAnthropic(choice.FinishReason)
31 }
32 }
33
34 cachedTokens := 0
35 if resp.Usage.PromptTokensDetails != nil {
36     cachedTokens = resp.Usage.PromptTokensDetails.CachedTokens
37 }
38
39 return &AnthropicResponse{
40     ID:        resp.ID,
41     Type:      "message",
42     Role:      "assistant",
43     Model:     resp.Model,
44     Content:   contentBlocks,
45     StopReason: stopReason,
46     Usage: AnthropicUsage{
47         InputTokens:      resp.Usage.PromptTokens - cachedTokens,
48         OutputTokens:      resp.Usage.CompletionTokens,
49         CacheReadInputTokens: cachedTokens,
50     },
51 }
52 }
53
54 // mapOpenAISTopReasonToAnthropic Stop Reason 映射
55 func mapOpenAISTopReasonToAnthropic(reason string) string {
56     switch reason {
57     case "stop":
58         return "end_turn"
59     case "length":
60         return "max_tokens"
61     case "tool_calls":

```

```
62         return "tool_use"
63     case "content_filter":
64         return "end_turn"
65     default:
66         return "end_turn"
67 }
68 }
```

4.9 流式响应状态机转换

流式模式下需要将 OpenAI SSE chunks 转换为 Anthropic SSE 事件序列，这需要一个有状态的转换器：

OpenAI chunk → Anthropic 事件映射：

Code block

```
1  第一个 chunk 到达（无论内容）：
2    → 发出 message_start（含 message 元数据、usage 初始值）
3
4  chunk.delta.content 有值：
5    → 若当前有 tool_use block 开启 → 先发 content_block_stop
6    → 若文本 block 未开启 → 发 content_block_start (type: "text")
7    → 发 content_block_delta (type: "text_delta", text: delta.content)
8
9  chunk.delta.tool_calls 有值（新工具调用）：
10   → 若有 block 开启 → 先发 content_block_stop
11   → 发 content_block_start (type: "tool_use", id, name, input: {})
12   → 开始接收 content_block_delta (type: "input_json_delta", partial_json)
13
14  chunk.choices[0].finish_reason 有值：
15   → 关闭所有开启的 block → 发 content_block_stop
16   → 发 message_delta (stop_reason, usage)
17   → 发 message_stop
18
19 收到 [DONE]：
20   → 结束流
```


Code block

```
1 // relay/channel/copilot/stream_translator.go
2
3 package copilot
4
5 // AnthropicStreamState 流式转换状态机
6 type AnthropicStreamState struct {
7     MessageStartSent bool
8     ContentBlockIndex int
9     ContentBlockOpen bool
10    // key: OpenAI tool_call index, value: 对应的 Anthropic block index
11    ToolCallBlocks map[int]ToolCallBlock
12 }
13
14 type ToolCallBlock struct {
15     ID string
16     Name string
17     AnthropicBlockIndex int
18 }
19
20 // TranslateChunkToAnthropicEvents 将单个 OpenAI chunk 转换为一组 Anthropic 流事件
21 func TranslateChunkToAnthropicEvents(
22     chunk *OpenAIChatChunk,
23     state *AnthropicStreamState,
24 ) []AnthropicStreamEvent {
25     var events []AnthropicStreamEvent
26
27     if len(chunk.Choices) == 0 {
28         return events
29     }
30     choice := chunk.Choices[0]
31     delta := choice.Delta
32
33     // 1. 首次 chunk: 发出 message_start
34     if !state.MessageStartSent {
35         cachedTokens := 0
36         if chunk.Usage != nil && chunk.Usage.PromptTokensDetails != nil {
37             cachedTokens = chunk.Usage.PromptTokensDetails.CachedTokens
38         }
39         promptTokens := 0
40         if chunk.Usage != nil {
41             promptTokens = chunk.Usage.PromptTokens
42         }
```

```

43     events = append(events, AnthropicStreamEvent{
44         Type: "message_start",
45         Message: &AnthropicMessageStart{
46             ID: chunk.ID,
47             Type: "message",
48             Role: "assistant",
49             Model: chunk.Model,
50             Usage: AnthropicUsage{
51                 InputTokens: promptTokens - cachedTokens,
52                 OutputTokens: 0,
53                 CacheReadInputTokens: cachedTokens,
54             },
55         },
56     })
57     state.MessageStartSent = true
58 }
59
60 // 2. 处理文本 delta
61 if delta.Content != "" {
62     if isToolBlockOpen(state) {
63         events = append(events, AnthropicStreamEvent{
64             Type: "content_block_stop",
65             Index: state.ContentBlockIndex,
66         })
67         state.ContentBlockIndex++
68         state.ContentBlockOpen = false
69     }
70     if !state.ContentBlockOpen {
71         events = append(events, AnthropicStreamEvent{
72             Type: "content_block_start",
73             Index: state.ContentBlockIndex,
74             ContentBlock: &AnthropicContentBlock{Type: "text", Text: ""},
75         })
76         state.ContentBlockOpen = true
77     }
78     events = append(events, AnthropicStreamEvent{
79         Type: "content_block_delta",
80         Index: state.ContentBlockIndex,
81         Delta: &AnthropicDelta{Type: "text_delta", Text: delta.Content},
82     })
83 }
84
85 // 3. 处理工具调用 delta
86 for _, tc := range delta.ToolCalls {
87     if tc.ID != "" && tc.Function.Name != "" {
88         // 新工具调用开始
89         if state.ContentBlockOpen {

```

```

90         events = append(events, AnthropicStreamEvent{
91             Type: "content_block_stop",
92             Index: state.ContentBlockIndex,
93         })
94         state.ContentBlockIndex++
95         state.ContentBlockOpen = false
96     }
97     blockIndex := state.ContentBlockIndex
98     state.ToolCallBlocks[tc.Index] = ToolCallBlock{
99         ID: tc.ID,
100        Name: tc.Function.Name,
101        AnthropicBlockIndex: blockIndex,
102    }
103    events = append(events, AnthropicStreamEvent{
104        Type: "content_block_start",
105        Index: blockIndex,
106        ContentBlock: &AnthropicContentBlock{
107            Type: "tool_use",
108            ID: tc.ID,
109            Name: tc.Function.Name,
110            Input: map[string]interface{}{},
111        },
112    })
113    state.ContentBlockOpen = true
114 }
115 if tc.Function.Arguments != "" {
116     if block, ok := state.ToolCallBlocks[tc.Index]; ok {
117         events = append(events, AnthropicStreamEvent{
118             Type: "content_block_delta",
119             Index: block.AnthropicBlockIndex,
120             Delta: &AnthropicDelta{
121                 Type: "input_json_delta",
122                 PartialJSON: tc.Function.Arguments,
123             },
124         })
125     }
126 }
127 }
128
129 // 4. 处理结束
130 if choice.FinishReason != "" {
131     if state.ContentBlockOpen {
132         events = append(events, AnthropicStreamEvent{
133             Type: "content_block_stop",
134             Index: state.ContentBlockIndex,
135         })
136         state.ContentBlockOpen = false

```

```

137     }
138
139     completionTokens := 0
140     promptTokens := 0
141     cachedTokens := 0
142     if chunk.Usage != nil {
143         completionTokens = chunk.Usage.CompletionTokens
144         promptTokens = chunk.Usage.PromptTokens
145         if chunk.Usage.PromptTokensDetails != nil {
146             cachedTokens = chunk.Usage.PromptTokensDetails.CachedTokens
147         }
148     }
149
150     events = append(events,
151         AnthropicStreamEvent{
152             Type: "message_delta",
153             Delta: &AnthropicDelta{
154                 StopReason:
155 mapOpenAISTopReasonToAnthropic(choice.FinishReason),
156                 StopSequence: nil,
157                 Usage: &AnthropicUsage{
158                     InputTokens:      promptTokens - cachedTokens,
159                     OutputTokens:      completionTokens,
160                     CacheReadInputTokens: cachedTokens,
161                 },
162             },
163             AnthropicStreamEvent{Type: "message_stop"},
164         )
165     }
166
167     return events
168 }
169
170 func isToolBlockOpen(state *AnthropicStreamState) bool {
171     if !state.ContentBlockOpen {
172         return false
173     }
174     for _, block := range state.ToolCallBlocks {
175         if block.AnthropicBlockIndex == state.ContentBlockIndex {
176             return true
177         }
178     }
179     return false
180 }

```

4.10 动态模型列表与 max_tokens 填充

动态模型列表（copilot-api 方案）：

Code block

```
1  // relay/channel/copilot/model.go
2
3  type CopilotModelsCache struct {
4      Models []CopilotModel
5      CachedAt time.Time
6  }
7
8  var modelsCache sync.Map // key: baseUrl
9
10 // FetchCopilotModels 从 Copilot /models 接口动态拉取 (缓存 30 分钟)
11 func FetchCopilotModels(baseUrl, copilotToken string) ([]CopilotModel, error) {
12     if cached, ok := modelsCache.Load(baseUrl); ok {
13         c := cached.(*CopilotModelsCache)
14         if time.Since(c.CachedAt) < 30*time.Minute {
15             return c.Models, nil
16         }
17     }
18
19     req, _ := http.NewRequest("GET", baseUrl+"/models", nil)
20     req.Header.Set("Authorization", "Bearer "+copilotToken)
21     InjectCopilotIdeHeaders(req, GetVSCodeVersion(), false, false)
22
23     resp, err := http.DefaultClient.Do(req)
24     if err != nil {
25         return nil, err
26     }
27     defer resp.Body.Close()
28
29     var result struct {
30         Data []CopilotModel `json:"data"`
31     }
32     json.NewDecoder(resp.Body).Decode(&result)
33
34     modelsCache.Store(baseUrl, &CopilotModelsCache{
```

```

35     Models:    result.Data,
36     CachedAt: time.Now(),
37 })
38     return result.Data, nil
39 }
40
41 // CopilotModel Copilot /models 返回的模型结构
42 type CopilotModel struct {
43     ID            string          `json:"id"`
44     Name          string          `json:"name"`
45     Vendor        string          `json:"vendor"`
46     Capabilities  CopilotCapabilities `json:"capabilities"`
47 }
48
49 type CopilotCapabilities struct {
50     Limits    CopilotLimits `json:"limits"`
51     Supports  CopilotSupports `json:"supports"`
52     Family    string        `json:"family"`
53     Type      string        `json:"type"`
54 }
55
56 type CopilotLimits struct {
57     MaxContextWindowTokens int `json:"max_context_window_tokens"`
58     MaxOutputTokens        int `json:"max_output_tokens"`
59     MaxPromptTokens        int `json:"max_prompt_tokens"`
60 }
61
62 type CopilotSupports struct {
63     ToolCalls        bool `json:"tool_calls"`
64     ParallelToolCalls bool `json:"parallel_tool_calls"`
65 }

```

动态填充 max_tokens:

Code block

```

1 // 在转发请求前, 如果 max_tokens 未设置, 从模型 capabilities 动态填充
2 // 避免 Anthropic 客户端 (如 Claude Code) 因 max_tokens 缺失而报错
3 func FillMaxTokens(payload *OpenAIChatRequest, models []CopilotModel) {
4     if payload.MaxTokens != nil && *payload.MaxTokens > 0 {
5         return
6     }
7     for _, m := range models {

```

```

8         if m.ID == payload.Model {
9             maxTokens := m.Capabilities.Limits.MaxOutputTokens
10            if maxTokens > 0 {
11                payload.MaxTokens = &maxTokens
12            }
13            return
14        }
15    }
16    // fallback: 如果找不到模型信息, 使用默认值
17    defaultMax := 8192
18    payload.MaxTokens = &defaultMax
19 }

```

4.11 new-api 渠道集成结构

目录结构:

Code block

```

1  relay/channel/copilot/
2  |— adaptor.go          # 实现 ChannelAdaptor 接口, 核心路由分发
3  |— token.go            # GitHub Token → Copilot Token exchange + 主动刷新
   缓存
4  |— headers.go          # 完整 Copilot IDE Headers 注入 (含 x-request-id
   等)
5  |— vscode_version.go    # 动态 VSCode 版本获取 (AUR + 缓存)
6  |— anthropic_to_openai.go # Anthropic Messages 请求 → OpenAI 格式 (含模型名规
   范化)
7  |— openai_to_anthropic.go # OpenAI 响应 → Anthropic Messages 格式
8  |— stream_translator.go # OpenAI SSE chunks → Anthropic SSE 事件 (状态机)
9  |— model.go            # 动态模型列表拉取、max_tokens 填充、模型名规范化
10 |— types.go             # 所有类型定义

```

注册新渠道类型 (参考 new-api 现有代码) :

```

1 // relay/channel/apitype/constant.go
2 const (
3     // ... 现有类型 ...
4     GitHubCopilot = 45
5 )
6
7 // relay/channel/helper.go
8 case apitype.GitHubCopilot:
9     return &copilot.Adaptor{}
10
11 // model/channel.go (渠道显示名称)
12 case 45:
13     return "GitHub Copilot"

```

五、内置模型列表

参考 copilot-api 动态从 `/models` 拉取的结果，以下为截至 2026 年 4 月的实测可用模型：

Code block

```

1 // relay/channel/copilot/model.go
2
3 var BuiltinModelList = []string{
4     // ===== OpenAI 系列 (直接透传 /chat/completions) =====
5     "gpt-4.1",
6     "gpt-4.1-mini",
7     "gpt-4.1-nano",
8     "gpt-4o",
9     "gpt-4o-mini",
10    "o3",
11    "o3-mini",
12    "o4-mini",
13
14    // ===== Claude 系列 (走 /chat/completions, 支持 /messages 双向转换) =====
15    "claude-opus-4",
16    "claude-opus-4-5",
17    "claude-sonnet-4",
18    "claude-sonnet-4-5",

```



```
19     "claude-haiku-4-5",
20
21     // ===== Gemini 系列 (走 /chat/completions) =====
22     "gemini-2.5-pro",
23     "gemini-2.5-flash",
24     "gemini-2.0-flash",
25
26     // ===== 其他 =====
27     "xai/grok-3",
28     "xai/grok-3-mini",
29     "mistral-large",
30     "cohere-command-r-plus",
31 }
```

推荐：启用后通过 `GET {baseUrl}/models` 动态拉取并更新列表，可自动适配不同 GitHub Plan 的可用模型。

六、实施优先级路线图

Phase 1 — MVP（预计 2 天）

Code block

- ✓ GitHub Token 配置 → Token Exchange (含 refresh_in 主动刷新)
- ✓ 完整 Headers 注入 (含 x-request-id、x-vscode-user-agent-library-version)
- ✓ GPT/o 系列透传 (OpenAI Chat Completions, 最简单)
- ✓ 模型名规范化 (Claude 子版本截断)
- ✓ 基础流式 SSE 支持
- ✓ new-api 渠道类型注册与 UI 展示

Phase 2 — Claude 支持（预计 2 天）

- Code block
- 1 ☒ /v1/messages 端点 (Anthropic 格式接入)
 - 2 ☒ Anthropic → OpenAI 请求转换 (含 tool_use、tool_result、thinking、image)
 - 3 ☒ OpenAI → Anthropic 响应转换 (非流式)
 - 4 ☒ 流式状态机转换 (OpenAI chunks → Anthropic SSE 事件序列)
 - 5 ☒ max_tokens 动态填充 (从模型 capabilities 读取)

Phase 3 — 增强功能 (预计 2-3 天)

Code block

- 1 ☒ 动态 VSCode 版本获取 (从 AUR 实时拉取, 24h 缓存)
- 2 ☒ 动态模型列表 (从 Copilot /v1/models 拉取, 30min 缓存)
- 3 ☒ 多账号类型支持 (individual/business/enterprise Base URL 自动选择)
- 4 ☒ X-Initiator Header 动态判断 (user vs agent)
- 5 ☒ 图片输入支持 (检测 image_url + copilot-vision-request 头)
- 6 ☒ Embedding 接口支持 (/v1/embeddings 透传)
- 7 ☒ Token 使用量查询 (api.github.com/copilot_internal/user)

Phase 4 — 生产优化 (预计 1-2 天)

Code block

- 1 ☒ Redis Token 缓存 (多实例部署共享, 替代内存缓存)
- 2 ☒ 多 GitHub Token 池 (多账号负载均衡 + 轮转)
- 3 ☒ 精细化错误处理 (429 限流重试、Token 过期立即刷新等)
- 4 ☒ 请求日志与可观测性 (Token 使用量、请求延迟监控)

七、关键注意事项

--	--	--

风险点	影响	处理方式
Copilot Token 过期	请求失败	使用 <code>refresh_in</code> 字段主动刷新，提前 60s 执行
IDE Headers 版本过时	全量 403	定期参考 cc-switch CHANGELOG 更新，或用 AUR 动态拉取方案
Claude 模型名含子版本号	400 参数错误	必须实现 <code>NormalizeCopilotModelName</code> 截断处理
<code>max_tokens</code> 未设置	Anthropic 客户端报错	从模型 capabilities 动态填充，fallback 8192
<code>proxy-ep</code> 因 Plan 不同而变化	请求打到错误端点	必须动态解析 ，不要硬编码 Base URL
<code>tool_result</code> 消息顺序	协议错误	<code>tool_result</code> 必须转换为 <code>role: "tool"</code> 且排在用户消息之前
<code>thinking</code> block 丢失	多轮对话上下文断裂	<code>thinking</code> block 必须合并进 <code>content</code> 文本传给 OpenAI
Copilot Internal API 变更	大规模失效	这是非公开 API，需定期跟踪三个参考项目的更新
单账号并发/限流	429 频率限制	Phase 4 实现多 Token 池 + 轮转策略
<code>x-request-id</code> 缺失	可能影响请求追踪	每次请求必须生成新 UUID

八、参考资料

项目	链接	主要参考点
copilot-api	ericc-ch/copilot-api	最佳参考 ：完整双向协议转换、refresh_in 主动刷新、动

		态 VSCode 版本、x-request-id、X-Initiator、模型名规范化、动态 max_tokens
openclaw	openclaw/openclaw	proxy-ep 动态 Base URL 解析、Token Exchange 实现、Claude Stream Headers
cc-switch	farion1231/cc-switch	IDE Headers 版本号跟踪、Copilot API 请求细节
copilot-api 核心文件	api-config.ts	完整 Headers 清单（含 x-vscode-user-agent-library-version）
copilot-api 协议转换	non-stream-translation.ts	Anthropic ↔ OpenAI 双向转换完整实现
copilot-api 流式转换	stream-translation.ts	OpenAI chunks → Anthropic SSE 状态机
copilot-api VSCode 版本	get-vscode-version.ts	